UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Facultad: por confirmar con el equipo

Escuela profesional: denominación oficial por confirmar

Escudo UNSAAC

Incorporar archivo institucional autorizado en `imagenes/escudo-unsaac.png`

Sistema de Gestión de Capacitaciones

Paz Sur — Cusco

Informe de Ingeniería de Software I

Primer incremento y diseño del producto

Docente: Jisbaj Gamarra Salas.

Integrantes:

- Brenda Lucia Mayhuire Chacón.
- Adel Alejandro Merma Ccarhuarupay.
- Franshesco (apellidos por confirmar).
- Daniela (apellidos por confirmar).

Implementación: asistida por Codex; revisión humana del incremento pendiente de registro.

Cusco, Perú

12 de noviembre de 2026

Resumen

Este proyecto propone organizar la planificación y el seguimiento de las capacitaciones de la empresa comercial Paz Sur. El registro manual de asistencia y su eventual transcripción a hojas de cálculo dificultan la consulta y la verificación de información. El alcance candidato reúne 28 historias de usuario que deben contrastarse con el cliente.

El informe presenta un proceso iterativo, incremental y adaptativo, una arquitectura de monolito modular y el primer incremento de autenticación, usuarios, roles y planificación de capacitaciones. La base inspeccionada contenía documentación y no una aplicación ejecutable; por ello se conserva el contrato existente y se construye el incremento sin atribuirle una historia previa ficticia. La interfaz utiliza React y TypeScript; la API, Node.js y Express; la persistencia local, SQLite con migraciones. PostgreSQL es una opción de evolución pendiente de acordar.

Los resultados de ejecución se consignan en el capítulo de pruebas y en su tabla de evidencia. El diseño de participantes, sesiones, asistencia, resultados, reportes, certificados y catálogos no debe confundirse con funcionalidades terminadas. La revisión humana, la integración y la validación de reglas comerciales se registrarán cuando efectivamente ocurran.

Palabras clave: capacitación, requisitos, Scrum, Kanban, pruebas, arquitectura, UNSAAC.

Índice general

Resumen I

1. Introducción 1

1.1. Descripción del problema . . . 1
1.2. Objetivo general . . . 1
1.3. Objetivos específicos . . . 1
1.4. Alcance y límites del informe . . . 1
1.5. Organización del documento . . . 2

2. Contexto y fundamento técnico 3

2.1. Participantes y responsabilidades . . . 3
2.2. Diagnóstico del repositorio . . . 3
2.3. Restricciones e incertidumbres . . . 3
2.4. Fundamento teórico . . . 4

3. Proceso de desarrollo 5

3.1. Enfoque seleccionado . . . 5
3.2. Artefactos y eventos propuestos . . . 5
3.3. Flujo y límites de trabajo . . . 5
3.4. Definition of Done . . . 6
3.5. Métricas y uso de IA . . . 6

4. Requisitos y trazabilidad 7

4.1. Fuentes y estado de validación . . . 7
4.2. Catálogo de historias candidatas . . . 7
4.3. Las tres C e INVEST . . . 8
4.4. Ejemplo de aceptación . . . 9
4.5. Supuestos que requieren conversación . . . 9

5. Arquitectura y modelos 10

5.1. Monolito modular . . . 10
5.2. Modelo de datos V1 . . . 10

5.3. Modelo candidato del producto . . . 11
5.4. Flujo de estados . . . 11
5.5. Permisos y seguridad . . . 12
5.6. Patrones evaluados . . . 12

6. Planificación del primer incremento **13**
6.1. Referencia histórica y propuesta actual . . . 13
6.2. Objetivo verificable . . . 13
6.3. Entrega mínima y capacidad . . . 13
6.4. Evolución propuesta . . . 13

7. Implementación del primer incremento **15**
7.1. Lenguajes y herramientas . . . 15
7.2. Archivos y comunicación entre módulos . . . 15
7.3. Autenticación, usuarios y roles . . . 16
7.4. Registro y edición de capacitaciones . . . 16
7.5. Fragmentos del código real . . . 17
7.6. Ejecución reproducible . . . 20
7.7. Dificultades y pendientes . . . 20
7.8. Interfaz verificada en navegador . . . 21

8. Pruebas y evidencia **23**
8.1. Estrategia . . . 23
8.2. Casos prioritarios . . . 23
8.3. Registro de ejecución . . . 24
8.4. Prueba manual propuesta . . . 25
8.5. Límites de la verificación . . . 25

9. Estructura, mantenimiento y refactorización **26**
9.1. Situación inicial real . . . 26
9.2. Problema que se busca evitar . . . 26
9.3. Protocolo para una refactorización futura . . . 26
9.4. Strategy, Adapter y evolución . . . 27

10. Resultados y limitaciones **28**
10.1. Artefactos preparados . . . 28
10.2. Resultado esperado del flujo V1 . . . 28
10.3. Limitaciones del producto . . . 28
10.4. Limitaciones del entorno y del proceso . . . 29
10.5. Siguiente actividad recomendada . . . 29

11. Conclusiones **30**

A. Bitácora de desarrollo **32**

A.1. Registro de futuras entradas 33

B. Fuentes de los diagramas **34**

B.1. Arquitectura general 34

B.2. Casos de uso 35

B.3. Entidad–relación V1 35

B.4. Entidad–relación del producto candidato 37

B.5. Responsabilidades de módulos 39

B.6. Componentes 40

B.7. Registrar una capacitación 41

B.8. Inscribir un participante: diseño futuro 41

B.9. Registrar asistencia: diseño futuro 42

B.10. Estados 42

B.11. Roles y permisos 43

Índice de figuras

5.1. Arquitectura del primer incremento. No se representa un despliegue externo. . . 10
5.2. Transiciones provisionales V1. Finalizada y cancelada son inmutables. 11

7.1. Panel real de demostración en escritorio. 21
7.2. Panel real de demostración en un viewport móvil. 22

Índice de tablas

4.1. Historias de referencia y alcance de implementación 7

5.1. Relaciones principales del modelo físico V1 11

6.1. Paquetes de trabajo del primer incremento 14

7.1. Responsabilidades de archivos principales 15

8.1. Cobertura que debe verificarse para el incremento 23

8.2. Evidencia de la sesión del 12 de noviembre de 2026 24

A.1. Actividades documentadas en esta sesión 32

Capítulo 1

Introducción

1.1. Descripción del problema

Paz Sur es una empresa comercial ubicada en Cusco, Perú. Según la información proporcionada por el equipo, sus trabajadores reciben capacitaciones internas y de marcas proveedoras. Algunas actividades se vinculan a temporadas comerciales, como campaña escolar, Navidad y Día de la Madre. El registro de asistencia se realiza parcialmente de forma manual y puede transcribirse después a Excel.

Este procedimiento requiere organizar la capacitación, identificar participantes y conservar un historial consultable. No se han confirmado cantidades de trabajadores, sedes, capacitaciones, costos, tiempos de transcripción ni tasas de error. Por tanto, el informe no atribuye cifras o problemas adicionales a la empresa.

1.2. Objetivo general

Desarrollar progresivamente un sistema de gestión de capacitaciones que permita organizar, consultar y verificar información, con implementación ejecutable, persistencia, controles de acceso, pruebas y documentación defendible por el equipo.

1.3. Objetivos específicos

1. Inspeccionar el repositorio y separar hechos, supuestos y propuestas.
2. Formular requisitos verificables y un backlog candidato trazable.
3. Diseñar una arquitectura coherente con los recursos de cuatro estudiantes.
4. Construir un primer flujo de extremo a extremo de acceso, usuarios, roles y capacitaciones.
5. Ejecutar verificaciones disponibles y registrar resultados sin fabricar evidencia.
6. Preparar el diseño y las prioridades de las iteraciones posteriores.

1.4. Alcance y límites del informe

El primer incremento aborda HU-01, HU-02, HU-03, HU-04 y HU-16 como dependencia de autenticación. El diseño contempla las 28 historias; no se declara implementado todo ese alcance.

Las reglas de asistencia, evaluaciones y certificados aún requieren validación. Un certificado puede existir en determinadas capacitaciones y no se presupone obligatorio.

Las versiones V1 a V4 expresan objetivos de evolución. No se crean etiquetas retroactivas para simular una secuencia histórica. El estado de terminado depende también de revisión humana e integración, además de pruebas locales.

1.5. Organización del documento

Los capítulos presentan contexto, proceso, requisitos, arquitectura, planificación, implementación, pruebas, evolución estructural y resultados. Los anexos conservan la bitácora y las fuentes Mermaid. Los fragmentos de código se importan de archivos reales del repositorio para reducir divergencias entre informe e implementación.

Capítulo 2

Contexto y fundamento técnico

2.1. Participantes y responsabilidades

El proyecto corresponde al curso Ingeniería de Software I de la UNSAAC, con el docente Jisbaj Gamarra Salas. El equipo está conformado por Brenda Lucia Mayhuire Chacón, Adel Alejandro Merma Ccarhuarupay, Franshesco y Daniela. No se completan apellidos, nombres institucionales de facultad o escuela, ni responsabilidades aceptadas mediante inferencias.

La coordinación propuesta debe acordar quién representa las decisiones del cliente, quién facilita el proceso y quién revisa cada entrega. La colaboración con IA no sustituye la responsabilidad de comprender y revisar el código. No se atribuye a un integrante una implementación manual o una aprobación que no haya realizado.

2.2. Diagnóstico del repositorio

La referencia inicial inspeccionada fue `cf6e720`, en la rama `main`. El árbol contenía README, el contrato de implementación y documentos de coordinación. No se encontraron la aplicación, un archivo de dependencias ni pruebas ejecutables en ese punto de partida.

Se observaron referencias remotas de integrantes: `origin/AD-1-registro-usuarios`, `origin/BrendaPro` y `origin/feature/brenda-mayhuire`. Su inspección no reveló una aplicación recuperable. Esta constatación no afirma que el código no exista en otras computadoras o referencias no disponibles.

Los documentos antiguos mencionan otra ruta local, una aplicación previa, SQLite y herramientas instaladas en esa otra situación. Se conservan como antecedentes y no como evidencia del entorno actual. La solicitud vigente autoriza construir el primer incremento; no se aplica automáticamente el antiguo límite de diagnóstico F0-01.

2.3. Restricciones e incertidumbres

- Reglas comerciales incompletas y textos originales de Jira no contrastados.
- Capacidad académica de cuatro estudiantes sin velocidad histórica medida.
- Prohibición de push, merge, borrados masivos y publicaciones sin autorización.
- No utilizar credenciales reales ni datos personales de la empresa como datos de demostración.

- Motor PostgreSQL y servicios Supabase/Render mencionados como intención, pero sin conexión o migración verificadas.
- Compilador LaTeX no encontrado en el primer diagnóstico; una comprobación posterior localizó MiKTeX. La compilación se registra de forma separada de la preparación de fuentes.

2.4. Fundamento teórico

Un requisito funcional describe una capacidad observable, como registrar una capacitación; un requisito no funcional expresa una cualidad o restricción, como evitar que una cuenta sin permiso edite registros. Una arquitectura separa responsabilidades para permitir cambios y pruebas sin concentrar todo el comportamiento en una pantalla.

Un incremento debe aportar un resultado verificable. Una iteración ofrece la oportunidad de inspeccionar y ajustar el trabajo. La combinación permite aprender de un flujo pequeño antes de extenderlo. Scrum aporta un marco de inspección y adaptación con objetivos y artefactos [7]. Kanban ayuda a definir y gestionar el flujo y sus límites [3]. Las prácticas de XP orientadas a pruebas y refactorización se seleccionan cuando ayudan a mantener el diseño y el comportamiento [1].

La trazabilidad conecta una necesidad con aceptación, código y verificación. No basta con que un nombre de historia aparezca en un archivo: se debe poder señalar la operación, los límites y el resultado comprobado.

Capítulo 3

Proceso de desarrollo

3.1. Enfoque seleccionado

Se propone un proceso iterativo, incremental y adaptativo. Cada Sprint de dos semanas debe tener un objetivo demostrable y un subconjunto de historias ajustado a la capacidad real. Scrum organiza la coordinación; Kanban hace visible el flujo; las prácticas técnicas de XP apoyan pruebas, refactorización e integración cuando existan las condiciones.

Esta es una adaptación para un trabajo académico, no una certificación de que ya se ejecutó Scrum. No se registran Daily Scrum, Sprint Review, retrospectivas, programación en pareja o TDD como actividades realizadas sin evidencia.

3.2. Artefactos y eventos propuestos

El Product Backlog mantiene el alcance candidato ordenado. El Sprint Backlog reúne el objetivo, historias seleccionadas y tareas. El incremento debe cumplir la Definition of Done acordada. Se propone una planificación inicial, un seguimiento breve y periódico, una revisión de producto y una retrospectiva al cierre; el equipo debe registrar fechas y resultados reales.

La guía detallada se conserva en `docs/PROCESO_DESARROLLO.md`. Los nombres de responsables propuestos no equivalen a una aceptación de sus funciones ni a una contribución ya realizada.

3.3. Flujo y límites de trabajo

Se plantea el tablero Pendiente, Lista, En curso, En revisión, Verificada y Hecha. Como política inicial para cuatro integrantes se recomienda una tarea de implementación activa por persona y un máximo de dos tareas en revisión. La coordinación puede reducir el WIP si aparecen bloqueos o archivos compartidos. Estos límites deben revisarse con observaciones del flujo, no tratarse como datos históricos.

Un bloqueo se anota con causa, responsable de resolverlo y siguiente acción. Si compromete el objetivo del Sprint se comunica al coordinador y se negocia alcance. No se declara terminada una tarea bloqueada por falta de acceso, revisión o integración.

3.4. Definition of Done

Una funcionalidad solo puede declararse terminada si cumple aceptación, valida entradas, persiste correctamente, aprueba las pruebas pertinentes, no tiene errores conocidos que impidan su flujo principal, cuenta con revisión humana registrada, está documentada y se integró en la rama acordada. La evidencia debe poder reproducirse.

Los criterios de una historia son específicos: por ejemplo, rechazar un correo duplicado. La DoD es transversal: por ejemplo, exigir revisión de código y documentación. Una batería local aprobada no satisface por sí sola todos los puntos de la DoD.

3.5. Métricas y uso de IA

Se proponen tiempo de ciclo, WIP, tareas bloqueadas, defectos encontrados y porcentaje de aceptación verificada. No se inventa velocidad de Sprints anteriores. La estimación de puntos es relativa y debe contrastarse con el equipo.

La plataforma identifica al agente como Codex basado en GPT-6; no expone aquí una variante exacta verificable. Se registra asistencia de IA sin atribuir autoría humana exclusiva. Las personas revisan requisitos, decisiones, código y evidencia. Los resultados de herramientas deben conservarse, y ningún texto generado sustituye una aprobación de Paz Sur.

Capítulo 4

Requisitos y trazabilidad

4.1. Fuentes y estado de validación

El prompt del proyecto aporta 28 historias de referencia. Estas constituyen el alcance candidato; su correspondencia con claves, textos, estimaciones y criterios reales de Jira sigue pendiente. No se reemplazan las estimaciones históricas de Jira con los puntos propuestos en la documentación.

Las fichas completas están en `docs/BACKLOG.md`; los criterios verificables y sus pruebas, en `docs/CRITERIOS_ACEPTACION.md`; requisitos y supuestos, en `docs/REQUISITOS.md`; y el vínculo con implementación y evidencia, en `docs/TRAZABILIDAD.md`.

4.2. Catálogo de historias candidatas

Tabla 4.1: Historias de referencia y alcance de implementación

ID	Nombre	Estado de alcance
HU-01	Registrar usuarios	Incluida en V1
HU-02	Asignar roles y permisos	Incluida en V1
HU-03	Registrar capacitación	Incluida en V1
HU-04	Modificar capacitación	Incluida en V1
HU-05	Asignar instructor	Candidata posterior
HU-06	Registrar participantes	Candidata posterior
HU-07	Consultar capacitaciones asignadas	Parcial: consulta por instructor V1
HU-08	Registrar asistencia	Candidata posterior
HU-09	Registrar evaluaciones	Candidata posterior
HU-10	Registrar certificado	Candidata posterior
HU-11	Consultar capacitaciones inscritas	Candidata posterior
HU-12	Consultar resultados	Candidata posterior

ID	Nombre	Estado de alcance
HU-13	Consultar historial de capacitación	Candidata posterior
HU-14	Consultar reportes consolidados	Candidata posterior
HU-15	Consultar estado de las capacitaciones	Parcial: estado en V1
HU-16	Iniciar sesión	Dependencia incluida en V1
HU-17	Gestionar instructores	Candidata posterior
HU-18	Descargar certificado	Candidata posterior
HU-19	Gestionar cupos de capacitación	Parcial: capacidad en V1
HU-20	Registrar marcas proveedoras	Candidata posterior
HU-21	Gestionar campañas comerciales	Candidata posterior
HU-22	Gestionar trabajadores por sede y área	Candidata posterior
HU-23	Consultar trabajadores pendientes	Candidata posterior
HU-24	Consultar capacitaciones por marca	Candidata posterior
HU-25	Consultar capacitaciones por sede	Candidata posterior
HU-26	Recibir alertas de programación	Candidata posterior
HU-27	Reprogramar capacitaciones pendientes	Parcial: edición en V1
HU-28	Registrar sesiones de capacitación	Candidata posterior

Que una capacitación permita un instructor opcional, muestre estado o tenga capacidad no acredita el cierre de HU-05, HU-15, HU-19 o HU-27 completas. Es necesario contrastar los criterios de cada historia y demostrar todos sus casos.

4.3. Las tres C e INVEST

Cada historia se formula como tarjeta con rol, necesidad y beneficio; la conversación registra incertidumbres y alternativas; la confirmación establece criterios verificables. INVEST se usa para revisar independencia, posibilidad de negociación, valor, estimación, tamaño y capacidad de prueba.

Las dependencias reales no se ocultan para aparentar independencia absoluta. Por ejemplo, registrar asistencia requiere una sesión y una inscripción válidas. Historias grandes como reportes o gestión de trabajadores deben dividirse en entregas pequeñas una vez acordadas las consultas y reglas prioritarias.

4.4. Ejemplo de aceptación

Para HU-03, un usuario autorizado que envía código único, título, periodo válido y capacidad positiva debe obtener una capacitación persistida. Un periodo invertido, un cupo cero o un estado inicial finalizada se rechazan. El código duplicado debe producir un conflicto comprensible y no una segunda fila. Un participante no obtiene permiso de escritura por enviar una solicitud manual.

Estos casos son específicos de HU-03 y se complementan con la DoD general. El registro de su ejecución pertenece al capítulo de pruebas, no a la definición del requisito.

4.5. Supuestos que requieren conversación

Se deben confirmar roles y alcance de RR. HH./gerencia, trabajadores sin cuenta, sedes y áreas, transiciones y reapertura, forma de evaluación, umbrales de asistencia, emisión opcional de certificados, reprogramación y destinatarios de alertas. Las cifras 80 % y 14/20 provienen del contrato, no de una validación documentada con el cliente.

Capítulo 5

Arquitectura y modelos

5.1. Monolito modular

Se separan presentación, entrada HTTP, lógica de negocio y persistencia. Un solo proceso backend y un solo motor de datos simplifican el arranque y las pruebas. La decisión completa se documenta en `docs/ARQUITECTURA.md` y `docs/DECISIONES_TECNICAS.md`.

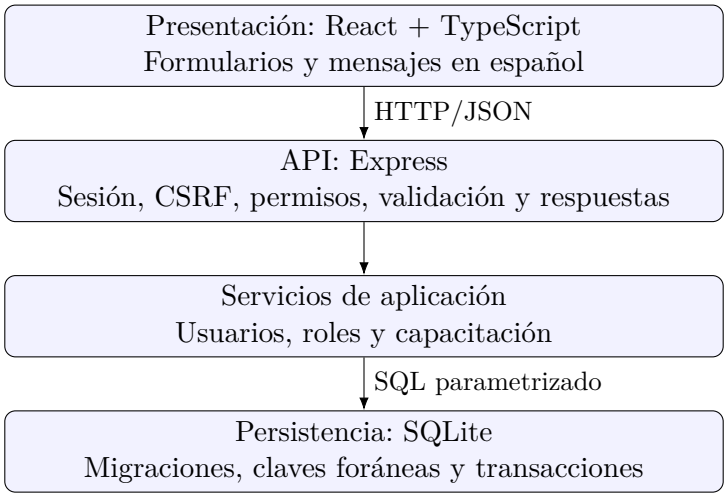


Figura 5.1: Arquitectura del primer incremento. No se representa un despliegue externo.

5.2. Modelo de datos V1

La base contiene usuarios, roles, permisos, asignaciones, sesiones de autenticación, capacitaciones, auditoría y control de migraciones. Una sesión de autenticación no es una sesión de capacitación: la primera mantiene el acceso; la segunda será una unidad académica en un incremento posterior.

Las tablas `user_roles` y `role_permissions` resuelven relaciones de muchos a muchos sin repetir listas de texto en una columna. El correo y el código de capacitación son únicos sin distinguir mayúsculas; el documento es opcional y único cuando existe. Una capacitación tiene responsable y puede tener instructor. El cupo admite de 1 a 10 000 personas.

Las migraciones se registran y aplican transaccionalmente. Los roles y permisos iniciales se cargan solo durante la primera migración; abrir la base o añadir otra migración no sobreescribe

las personalizaciones administrativas. Los timestamps técnicos usan UTC y el panel determina el día actual en `America/Lima`.

Tabla 5.1: Relaciones principales del modelo físico V1

Relación	Integridad
Usuario–rol	PK compuesta impide asignación duplicada
Rol–permiso	FK a catálogos conocidos y combinación única
Usuario–sesión de acceso	Token hasheado, vencimiento y usuario válido
Usuario–capacitación	Responsable obligatorio; instructor opcional
Usuario–auditoría	Actor de una acción, sin contraseñas o cookies

5.3. Modelo candidato del producto

El diseño posterior contempla trabajador, sede, área, marca proveedora, campaña, sesión académica, inscripción, asistencia, evaluación, certificado y notificación. No se crean entidades por acumular nombres: un instructor V1 es un usuario con rol instructor, sin duplicar su identidad en una tabla independiente.

Cada inscripción futura vincula persona y capacitación, con unicidad de esa pareja. Cada marca de asistencia vincula inscripción y sesión y debe comprobar que ambas pertenecen a la misma capacitación. Evaluación y certificado son opcionales por inscripción. Las alertas se plantean dentro de la aplicación; no se asume un proveedor de correo o SMS.

La alternativa candidata del contrato exige una cuenta para inscribirse, aunque un perfil de trabajador pueda existir sin acceso. Si se requiere inscribir a trabajadores sin cuenta, la FK deberá apuntar al trabajador y se revisarán contrato, permisos y migración antes de implementar. El diagrama señala esta incertidumbre en lugar de simular una solución aprobada.

5.4. Flujo de estados

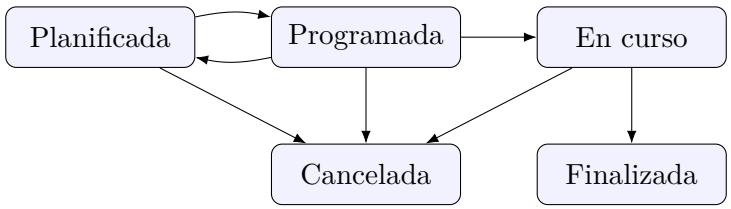


Figura 5.2: Transiciones provisionales V1. Finalizada y cancelada son inmutables.

La creación admite planificada o programada. La edición puede conservar un estado abierto. No se permite saltar directamente de planificada a finalizada. La reapertura no forma parte de este incremento y requiere una decisión del cliente.

5.5. Permisos y seguridad

La API comprueba permisos en el servidor y filtra los datos por alcance. Los seis roles iniciales son administrador, coordinador, instructor, participante, RR. HH. y gerencia. El administrador tiene todas las capacidades y su rol se protege; los demás roles parten de la matriz del contrato y sus permisos pueden editarse con autorización.

Asignar roles requiere `roles.manage`. Una cuenta delegada con solo `users.manage` crea únicamente participantes y no edita cuentas que tengan privilegios `users.manage` o `roles.manage`. El permiso `people.manage` habilita en V1 consultas reducidas para selectores, sin correo, documento, roles o permisos ajenos; las mutaciones de gestión de instructores y participantes continúan pendientes.

La contraseña se protege con `scrypt` y una sal propia; el navegador recibe una cookie HttpOnly con token opaco y la base almacena su hash. La sesión vence en ocho horas y se revoca al desactivar la cuenta o cambiar la contraseña. Las mutaciones requieren un token CSRF. Se limitan diez intentos de login por IP en quince minutos, incluidos los correctos y sin reiniciar contador al acertar. El control reside en memoria de una sola instancia y se pierde al reiniciar.

La explicación de estas primitivas se apoya en la documentación de Node [4] y las recomendaciones de OWASP [6]. Un despliegue real requiere TLS, cookie Secure y revisar sus condiciones de operación.

5.6. Patrones evaluados

Strategy no se aplica todavía porque no hay varias políticas de evaluación aprobadas. Adapter se reconsiderará cuando exista un proveedor externo o una diferencia real de integración. Un mapa central de transiciones resuelve los estados V1 sin una jerarquía de clases State. Se documentan las condiciones de aplicación sin fabricar un problema para justificar un patrón.

El diagrama de clases del anexo representa responsabilidades conceptuales de módulos. La implementación backend utiliza funciones; no se afirma que esas clases existan literalmente en JavaScript.

Capítulo 6

Planificación del primer incremento

6.1. Referencia histórica y propuesta actual

La referencia histórica proporcionada por el usuario sitúa HU-01 a HU-04 en Sprint 1. No se han consultado las estimaciones originales de Jira y no se modifican. HU-16 se identifica como dependencia técnica y funcional del acceso protegido, sin afirmar que estuviera incluida históricamente.

La planificación propuesta se conserva en `docs/SPRINT_1.md`. La duración es dos semanas, pero no se inventan fechas de eventos, horas disponibles o velocidad del equipo. Antes de comprometer todo el alcance se deben contrastar disponibilidad, dificultad y criterios con los cuatro integrantes.

6.2. Objetivo verificable

Una cuenta administradora de demostración puede iniciar sesión, registrar una cuenta con un rol permitido, crear una capacitación, editarla y volver a consultar los datos persistidos. Las solicitudes sin acceso, los datos inválidos y los conflictos reciben errores comprensibles; el navegador no simula un guardado local.

6.3. Entrega mínima y capacidad

Si no existe capacidad para terminar todo en el Sprint, la entrega mínima debe contener login, un usuario autorizado y creación/consulta persistente de capacitación, con aceptación y pruebas. Edición completa de permisos u otras tareas deben registrarse pendientes, sin declarar terminado un incremento que no cumple su objetivo.

La documentación puede elaborarse en paralelo con límites de archivos claros, pero una revisión de código por otro integrante sigue siendo necesaria. Las tareas compartidas deben integrarse de forma consciente; las contribuciones de IA no representan automáticamente trabajo de cada estudiante.

6.4. Evolución propuesta

V2 revisará el diseño real y ampliará instructores y participantes cuando sus dependencias estén listas. V3 podrá incorporar sesiones, asistencia, historial y evaluaciones. V4 priorizará integra-

Tabla 6.1: Paquetes de trabajo del primer incremento

Referencia	Resultado esperado	Verificación
HU-16	Login, sesión vigente y logout	API y revisión de navegador
HU-01	Alta, consulta y edición de usuarios	Validez, duplicado y acceso
HU-02	Roles, asignaciones y permisos	Privilegios y administrador protegido
HU-03	Crear capacitación persistente	Caso válido y límites
HU-04	Editar sin violar integridad	Consulta posterior y estados
Técnica	Migraciones, semilla, documentación	Arranque reproducible

ción, seguridad, experiencia y funcionalidades restantes acordadas. Ninguna de estas metas se etiqueta como versión ya construida o probada.

Capítulo 7

Implementación del primer incremento

7.1. Lenguajes y herramientas

La interfaz utiliza TypeScript con React. La API utiliza JavaScript ESM ejecutado con Node.js 24 y Express 5. Zod valida entradas con mensajes en español; SQLite persiste datos mediante el módulo `node:sqlite`; Node Test Runner y Supertest verifican la API, y Playwright automatiza flujos de navegador Chrome.

Las versiones resueltas en esta construcción son Express 5.3.0, React 19.3.1, Zod 4.6.7, Vite 8.3.8 y TypeScript 7.0.2. La fuente reproducible es `package-lock.json`; estas versiones no estaban presentes en la base documental inicial. El entorno inspeccionado ofrece Node 24.16.0. La documentación oficial describe los requisitos y las funciones de Express, Vite y SQLite [2, 8, 5].

7.2. Archivos y comunicación entre módulos

Tabla 7.1: Responsabilidades de archivos principales

Archivo o carpeta	Responsabilidad
`src/App.tsx` y módulos de interfaz	Formularios y vistas de acceso, panel, usuarios, roles y capacitaciones
`src/api.ts`	Solicitudes a la API y tratamiento de errores de contrato
`server/index.js`	Inicio del proceso y configuración del entorno
`server/app.js`	Composición de middleware, rutas y manejo final de errores
`server/routes/auth.js`	Inicio, consulta y cierre de sesión
`server/routes/users.js`	Entradas HTTP de usuarios y roles
`server/routes/trainings.js`	Entradas HTTP de capacitaciones y consultas iniciales
`server/services/user-service.js`	Reglas de usuarios, roles e integridad administrativa
`server/services/training-service.js`	Validación de planificación y transiciones

Archivo o carpeta	Responsabilidad
`server/security.js`	Hashing, sesiones, CSRF y autorización
`server/db.js`	Conexión, migraciones y transacciones
`server/migrations/001_initial.sql`	Esquema inicial con restricciones persistentes
`server/seed.js`	Datos ficticios de demostración y configuración de cuenta inicial
`tests/api.test.js`	Casos HTTP de usuarios, roles y capacitaciones
`tests/security.test.js`	Sesión, CSRF, límites y controles de privilegios
`tests/persistence.test.js`	Reapertura de base, rollback y migraciones incrementales
`tests/ui.spec.ts`	Flujos de navegador automatizados; resultados separados de API

El formulario no ejecuta SQL. Envía JSON a una ruta, que valida y delega al servicio. El servicio aplica reglas y coordina persistencia. La respuesta informa éxito después de confirmar la operación. Una falla de integridad no debe dejar la auditoría o una asignación de rol parcialmente guardadas.

7.3. Autenticación, usuarios y roles

HU-16 habilita el flujo protegido de HU-01 y HU-02. El login verifica la contraseña y el estado activo; la cookie de sesión y el token CSRF acompañan solicitudes posteriores. Un usuario autenticado no recibe automáticamente permiso administrativo.

El registro de usuario admite nombres, apellidos, correo, documento opcional, área textual, especialidad, estado y roles. La contraseña solo es obligatoria al crear y nunca se devuelve en consultas. Una edición debe conservar administración activa y rechazar duplicados. La asignación de roles exige `roles.manage`; delegar solo `users.manage` no permite crear cuentas privilegiadas ni editar las que tengan gestión de usuarios o roles. La gestión de roles permite consultar el catálogo y cambiar permisos de roles no administrativos con autorización.

7.4. Registro y edición de capacitaciones

HU-03 y HU-04 usan una capacitación con código, título, descripción, periodo, horario, modalidad, ubicación, capacidad y estado. El responsable procede de la sesión autenticada; no se confía en un identificador privilegiado enviado por el navegador. El instructor es opcional y, cuando se informa, debe ser un usuario válido con el rol correspondiente.

La lista permite consulta y filtros. La creación admite estados iniciales definidos y la edición comprueba el grafo de transiciones. El control de capacidad V1 valida un entero entre 1 y 10 000; aún no prueba un flujo de ocupación de cupos con inscripciones, porque ese módulo está pendiente. El instructor consulta únicamente capacitaciones asignadas. El participante no recibe filas ajenas mientras no exista el módulo de inscripciones.

7.5. Fragmentos del código real

Los siguientes listados se leen directamente del código del repositorio. Si se prepara una copia autónoma para Overleaf, el script incluido copia esos mismos archivos y registra sus hashes. No se presenta pseudocódigo como una captura de implementación.

```
import { DatabaseSync } from 'node:sqlite';
import { mkdirSync, readdirSync, readFileSync } from 'node:fs';
import { dirname, resolve } from 'node:path';
import { fileURLToPath } from 'node:url';
import { PERMISSIONS, ROLES } from './constants.js';

export const now = () => new Date().toISOString();

export function transaction(db, operation) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = operation();
    if (result?.then) throw new Error('Una transacción SQLite requiere una función síncrona.');
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}

export function createDatabase(path = process.env.DATABASE_PATH || './data/paz-sur.sqlite', {
   migrationsDirectory = fileURLToPath(new URL('./migrations/', import.meta.url)) } = {}) {
  if (path !== ':memory:') mkdirSync(dirname(resolve(path)), { recursive: true });
  const db = new DatabaseSync(path);
  try {
    db.exec('PRAGMA foreign_keys = ON; PRAGMA journal_mode = WAL; PRAGMA busy_timeout = 5000;')
    ;
    db.exec('CREATE TABLE IF NOT EXISTS schema_migrations (version TEXT PRIMARY KEY, applied_at
    TEXT NOT NULL) STRICT');
    const directory = migrationsDirectory;
    for (const version of readdirSync(directory).filter(name => name.endsWith('.sql')).sort())
    {
      if (db.prepare('SELECT 1 FROM schema_migrations WHERE version = ?').get(version))
    continue;
      transaction(db, () => {
        db.exec(readFileSync(resolve(directory, version), 'utf8'));
        if (version === '001_initial.sql') {
          for (const [code, name] of Object.entries(PERMISSIONS)) db.prepare('INSERT INTO
   permissions (code, name) VALUES (?, ?)').run(code, name);
          for (const role of ROLES) {
            const { lastInsertRowid } = db.prepare('INSERT INTO roles (code, name) VALUES (?,
   ?)').run(role.code, role.name);
            for (const permission of role.permissions) db.prepare('INSERT INTO role_permissions
    (role_id, permission_code) VALUES (?, ?)').run(lastInsertRowid, permission);
          }
        }
        db.prepare('INSERT INTO schema_migrations (version, applied_at) VALUES (?, ?)').run(
   version, now());
      });
    }
    return db;
  } catch (error) {
    db.close();
```

Listing 7.1: Inicio del módulo real de persistencia y migraciones

```
import { createHash, randomBytes, scrypt as scryptCallback, timingSafeEqual } from 'node:crypto';
import { promisify } from 'node:util';
import { HttpError } from './errors.js';
import { now } from './db.js';

const scrypt = promisify(scryptCallback);
export const tokenHash = value => createHash('sha256').update(value).digest('hex');
export const randomToken = () => randomBytes(32).toString('hex');
export async function hashPassword(password) {
  const salt = randomBytes(16).toString('hex');
  const derived = await scrypt(password, salt, 64, { N: 32768, r: 8, p: 3, maxmem: 64 * 1024 * 1024 });
  return `scrypt$32768$8$3$${salt}$${derived.toString('hex')}`;
}
export async function verifyPassword(password, hash) {
  const [algorithm, n, r, p, salt, digest] = hash.split('$');
  if (algorithm !== 'scrypt' || n !== '32768' || r !== '8' || p !== '3' || !/^[a-f0-9]{128}$/.test(digest || '')) return false;
  const derived = await scrypt(password, salt, 64, { N: Number(n), r: Number(r), p: Number(p), maxmem: 64 * 1024 * 1024 });
  return timingSafeEqual(derived, Buffer.from(digest, 'hex'));
}
export function hydrateUser(db, id) {
  const row = db.prepare('SELECT id, first_name, last_name, email, document, active, department, specialty, created_at, updated_at FROM users WHERE id = ?').get(id);
  if (!row) return null;
  const roles = db.prepare('SELECT r.code FROM roles r JOIN user_roles ur ON ur.role_id = r.id WHERE ur.user_id = ? ORDER BY r.code').all(id).map(item => item.code);
  const permissions = db.prepare('SELECT DISTINCT rp.permission_code FROM role_permissions rp JOIN user_roles ur ON ur.role_id = rp.role_id WHERE ur.user_id = ? ORDER BY rp.permission_code').all(id).map(item => item.permission_code);
  return { ...row, active: Boolean(row.active), roles, permissions };
}
export function sessionMiddleware(db) {
  return (req, _res, next) => {
    const raw = /(?:^|;\s*)sgc_session=([a-f0-9]{64})(?:;|$)/.exec(req.headers.cookie || '')?.[1];
    if (raw) {
      const session = db.prepare('SELECT * FROM sessions_auth WHERE token_hash = ? AND expires_at > ?').get(tokenHash(raw), now());
      if (session) {
        const user = hydrateUser(db, session.user_id);
        if (user?.active) { req.user = user; req.session = session; }
      }
    }
    next();
  };
}
export const requireAuth = (req, _res, next) => req.user ? next() : next(new HttpError(401, 'Inicie sesión para continuar.'));
export const requirePermission = (...permissions) => (req, _res, next) => {
  if (!req.user) return next(new HttpError(401, 'Inicie sesión para continuar.'));
  if (!permissions.some(permission => req.user.permissions.includes(permission))) return next(new HttpError(403, 'No tiene permiso para realizar esta acción.'));
  next();
};
export function csrfMiddleware(req, _res, next) {
  if (!['POST', 'PUT', 'PATCH', 'DELETE'].includes(req.method)) return next();
  if (!req.is('application/json')) return next(new HttpError(415, 'Envíe el contenido como application/json.'));
  if (!req.user) return next();
  const value = req.get('X-CSRF-Token') || '';
```

```
  const expected = req.session.csrf_token;
  if (!/^[a-f0-9]{64}$/.test(value) || !timingSafeEqual(Buffer.from(value), Buffer.from(
   expected))) return next(new HttpError(403, 'La verificación de seguridad caducó. Actualice
   la página.'));
  next();
}
export function audit(db, userId, action, entity, entityId) {
```

Listing 7.2: Inicio del módulo real de seguridad

```
import { z } from 'zod';
import { HttpError, validate, paginationSchema, paginated, searchPattern } from '../errors.js';
import { MODALITIES, TRAINING_STATUSES, TRANSITIONS } from '../constants.js';
import { now, transaction } from '../db.js';
import { audit, hydrateUser } from '../security.js';

const dateSchema = z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Use una fecha YYYY-MM-DD.').refine(
   value => {
  const date = new Date(`${value}T12:00:00Z`);
  return Number.isFinite(date.getTime()) && date.toISOString().slice(0, 10) === value;
}, 'La fecha no existe.');
const timeSchema = z.string().regex(/^(?:[01]\d|2[0-3]):[0-5]\d$/, 'Use una hora HH:mm.');
const schema = z.object({
  code: z.string().trim().min(1, 'Ingrese el código.').max(30),
  title: z.string().trim().min(3, 'Ingrese un título de al menos 3 caracteres.').max(160),
  description: z.string().trim().max(4000).default(''),
  start_date: dateSchema,
  end_date: dateSchema,
  start_time: timeSchema,
  end_time: timeSchema,
  modality: z.enum(MODALITIES),
  location: z.string().trim().min(1, 'Indique el lugar o enlace.').max(240),
  capacity: z.number().int().min(1, 'El cupo mínimo es 1.').max(10000),
  status: z.enum(TRAINING_STATUSES).default('planificada'),
  instructor_id: z.number().int().positive().nullable().default(null),
  // Se acepta por compatibilidad, pero siempre se conserva el responsable autenticado.
  responsible_id: z.number().int().positive().optional(),
}).strict().superRefine((value, context) => {
  if (value.end_date < value.start_date) context.addIssue({ code: 'custom', path: ['end_date'],
    message: 'La fecha final debe ser igual o posterior al inicio.' });
  if (value.end_time <= value.start_time) context.addIssue({ code: 'custom', path: ['end_time'
   ], message: 'La hora final debe ser posterior a la inicial (horario diario).' });
});
const select = `SELECT t.*, trim(i.first_name || ' ' || i.last_name) AS instructor_name, 0 AS
   enrolled_count, t.capacity AS available_seats FROM trainings t LEFT JOIN users i ON i.id=t.
   instructor_id`;
function accessFilter(user) {
  if (user.permissions.includes('trainings.view_all') || user.permissions.includes('trainings.
   manage')) return { where: '1=1', args: [] };
  if (user.permissions.includes('teaching.manage')) return { where: 't.instructor_id=?', args:
   [user.id] };
  // V1 aún no tiene inscripciones; el participante no obtiene datos de terceros.
  return { where: '0=1', args: [] };
}
export function getTraining(db, id, user) {
  const { where, args } = accessFilter(user);
  const row = db.prepare(`${select} WHERE t.id=? AND (${where})`).get(id, ...args);
  if (!row) throw new HttpError(404, 'La capacitación no existe o no está disponible para su
   cuenta.');
  return { ...row, sessions: [], enrollments: [] };
}
export function listTrainings(db, query, user) {
  const { page, pageSize, search, status, from, to } = validate(z.object({
    ...paginationSchema, search: z.string().trim().max(160).default(''),
```

```
    status: z.preprocess(value => value === '' ? undefined : value, z.enum(TRAINING_STATUSES).optional()),
    from: z.preprocess(value => value === '' ? undefined : value, dateSchema.optional()),
    to: z.preprocess(value => value === '' ? undefined : value, dateSchema.optional()),
  }).refine(value => !value.from || !value.to || value.from <= value.to, { path: ['to'], message: 'El rango de fechas está invertido.' }), query);
  const access = accessFilter(user);
  let where = `(${access.where}) AND (t.title LIKE ? ESCAPE '\\' OR t.code LIKE ? ESCAPE '\\')`;
  const args = [...access.args, searchPattern(search), searchPattern(search)];
  if (status) { where += ' AND t.status=?'; args.push(status); }
  if (from) { where += ' AND t.start_date>=?'; args.push(from); }
  if (to) { where += ' AND t.start_date<=?'; args.push(to); }
  const total = db.prepare(`SELECT COUNT(*) AS total FROM trainings t WHERE ${where}`).get(...args).total;
  const rows = db.prepare(`${select} WHERE ${where} ORDER BY t.start_date DESC,t.id DESC LIMIT ? OFFSET ?`).all(...args, pageSize, (page - 1) * pageSize);
  return paginated(rows, total, page, pageSize);
}
```

Listing 7.3: Inicio del servicio real de capacitaciones

7.6. Ejecución reproducible

Desde la raíz del proyecto, se instala con `npm.cmd ci`, se prepara `.env` desde `.env.example` y se definen credenciales ficticias propias para la demostración. Los valores reales no se incluyen en este informe. Después se aplican migraciones, se carga la semilla y se ejecuta el desarrollo:

```
npm.cmd run db:migrate
npm.cmd run db:seed
npm.cmd run dev
```

La dirección y las variables exactas se consultan en el README y en el archivo de ejemplo que acompaña al incremento. Para comprobar tipos, API y compilación:

```
npm.cmd run typecheck
npm.cmd test
npm.cmd run build
```

La prueba automatizada de interfaz requiere Chrome disponible y una compilación previa. Se ejecuta con `npm.cmd run test:ui`; su configuración inicia un servidor propio y una base ficticia aislada en el puerto 3101. No usa la base de demostración del usuario. Los resultados y las capturas reales, cuando se produzcan, se registran como evidencia separada.

El resultado observable esperado es iniciar sesión, guardar un usuario o una capacitación y recuperarlos de la API después de recargar. La prueba de persistencia en disco requiere volver a abrir la misma base; la presencia de una lista en pantalla por sí sola no demuestra ese comportamiento.

7.7. Dificultades y pendientes

Las guías iniciales no coincidían con el árbol real. Se decidió preservar los documentos y explicitar el nuevo punto de partida. Durante el trabajo se detectó falta de espacio en la unidad del perfil de usuario; las herramientas y el cache de instalación se dirigieron al espacio disponible del proyecto cuando fue necesario. Esto es una limitación del entorno y no una función del producto.

Los módulos de inscripción, asistencia, evaluación, historial, certificados, reportes consolidados, marcas, campañas, sedes y alertas quedan fuera del flujo principal V1. No se incorporan botones que aparenten guardar esas funciones.

7.8. Interfaz verificada en navegador

Las figuras se obtuvieron con Chrome y Playwright ejecutando la aplicación y su API aislada con datos ficticios. Son capturas reales del incremento; no representan aprobación del cliente ni revisión humana del equipo. Se comprobó también el guardado mediante formularios, el rechazo de correos duplicados y la consulta del detalle actualizado.

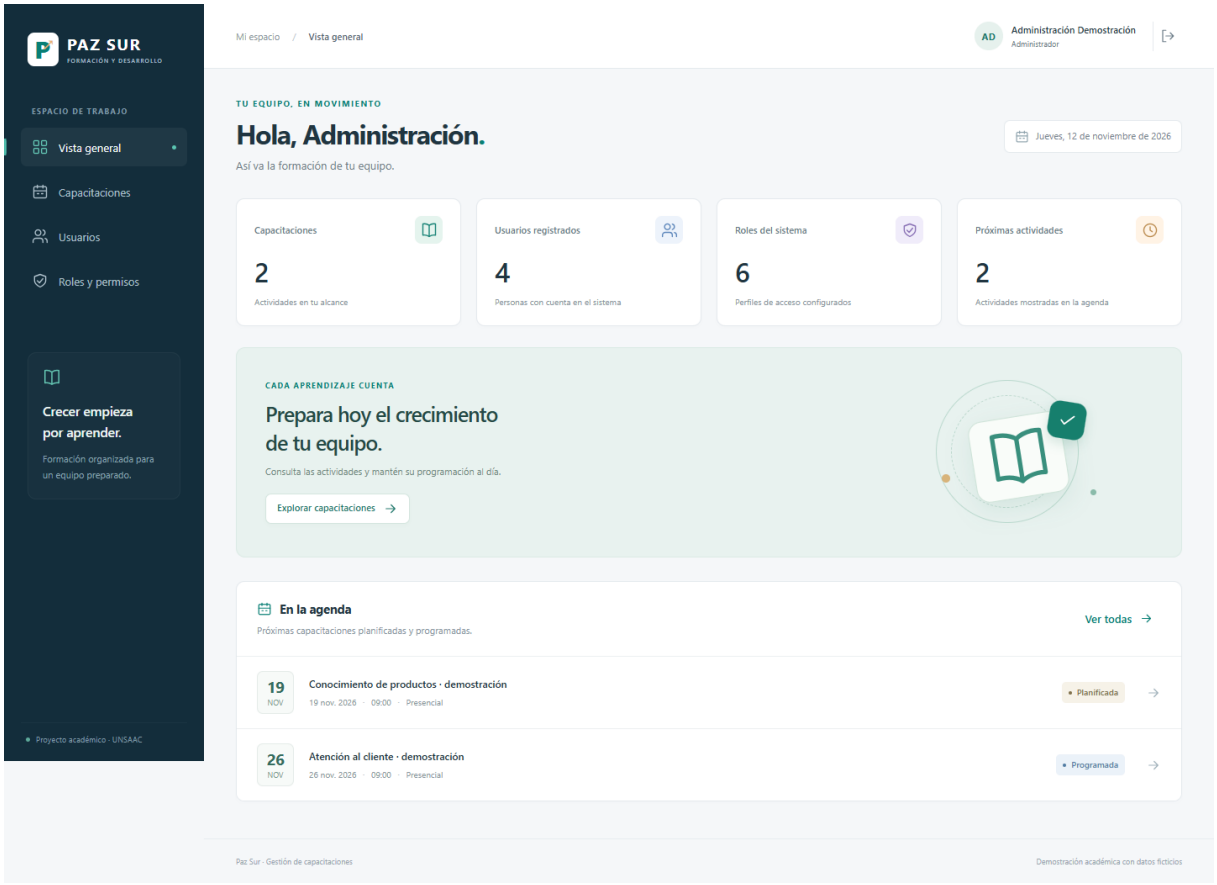


Figura 7.1: Panel real de demostración en escritorio.



Figura 7.2: Panel real de demostración en un viewport móvil.

Capítulo 8

Pruebas y evidencia

8.1. Estrategia

Se verifica la API real con Supertest y Node Test Runner. Los archivos `tests/api.test.js`, `tests/security.test.js` y `tests/persistence.test.js` utilizan bases aisladas y datos ficticios. Los casos cubren solicitudes válidas, entradas inválidas, límites y controles de acceso. Las consultas posteriores comprueban que la respuesta de éxito corresponde a datos guardados.

Playwright añade los flujos de `tests/ui.spec.ts` sobre Chrome y un servidor con base E2E propia. Es una automatización de navegador y no una revisión humana. Su ejecución se registra separadamente en la tabla de evidencia; escribir el archivo de prueba no demuestra que haya aprobado.

La documentación de aceptación define el resultado esperado; este capítulo registra lo observado. La definición de un caso no demuestra que haya sido ejecutado. Un análisis de tipos o una compilación frontend tampoco reemplazan pruebas de integración.

8.2. Casos prioritarios

Tabla 8.1: Cobertura que debe verificarse para el incremento

Área	Caso y resultado esperado	Relación
Acceso	Credenciales válidas crean sesión; contraseña inválida y cuenta inactiva se rechazan	HU-16
Sesión	Sin cookie no se accede a rutas protegidas; logout invalida la sesión	HU-16
CSRF	Una mutación autenticada sin token válido se rechaza	Seguridad
Usuarios	Alta válida y lectura posterior; duplicado de correo o documento rechazado	HU-01
Roles	Asignar roles exige roles.manage; users.manage delegado no eleva privilegios ni edita cuentas administrativas	HU-02
Capacitación	Registro válido, consulta y modificación persistidas	HU-03/04
Límites	Capacidad cero, fecha invertida, hora inválida y datos incompletos se rechazan	HU-03/04

Área	Caso y resultado esperado	Relación
Estado	Transición válida admitida; salto no permitido y edición de cierre rechazados	HU-04
Alcance	Instructor no consulta capacitaciones ajenas; participante no ve datos de otros	Acceso a datos
Persistencia	Reapertura conserva registros; rollback revierte todo; migración incremental conserva catálogos personalizados	Técnica

Los nombres y el número exacto de pruebas son los de los archivos y la salida del comando real. La tabla anterior no los sustituye ni promete cobertura de funcionalidades pendientes.

8.3. Registro de ejecución

Tabla 8.2: Evidencia de la sesión del 12 de noviembre de 2026

Comprobación	Resultado registrado
Inspección Git	Base `cf6e720`; rama local de incremento `feature/v1-base-funcional`. Sin commit, push, merge o tag de esta entrega acreditados.
Motor local	Node 24.16.0; apertura de SQLite en memoria devolvió versión 3.53.0.
Instalación	133 paquetes auditados después de incorporar Playwright; cero vulnerabilidades reportadas por npm en esa consulta. No equivale a una auditoría de seguridad.
`npm.cmd run check`	Verificación final: 28 pruebas aprobadas, 0 fallidas en 12.52 s; también análisis de tipos y build aprobados. Incluye fecha Lima y privilegios indirectos. Véase `docs/PRUEBAS.md`.
Tipos TypeScript	`tsc -noEmit` aprobado como paso de build. No se afirma ejecución separada del script typecheck.
`npm.cmd run build`	Aprobado con Vite 8.3.8, 22 módulos. El primer intento detectó falta de tipos del import CSS, corregida antes de la ejecución aprobada.
Prueba de navegador	Playwright y Chrome: 4 aprobadas, 0 fallidas en 9.0 s. Login/logout, alta/edición/duplicado de usuario, alta/edición/detalle de capacitación y panel móvil. Se corrigieron títulos que se cortaban en 390 px. Capturas reales en `docs/evidencias/`.
Compilación LaTeX	XeLaTeX falló por ausencia de xelatex.fmt. pdfLaTeX y BibTeX generaron el PDF tras corregir Babel/TikZ y usar fuentes Latin Modern. La inspección visual final se registra en `docs/PRUEBAS.md`.

Comprobación	Resultado registrado
Revisión humana e integración	PENDIENTES; no declarar cumplida la DoD completa sin estos registros.

8.4. Prueba manual propuesta

Con la semilla ficticia y la aplicación ejecutándose, el revisor puede iniciar sesión, crear un usuario, intentar repetir su correo, crear una capacitación, editar su título y volver a consultar. Debe observar guardado real y errores comprensibles. Luego puede comprobar que una cuenta sin privilegios no dispone de las mismas acciones ni obtiene acceso mediante llamadas directas.

Esta secuencia es un protocolo pendiente hasta que una persona registre fecha, cuenta ficticia usada, resultado y observaciones. No se atribuye una ejecución de navegador a los cuatro integrantes ni se presentan capturas generadas como evidencia.

8.5. Límites de la verificación

Las pruebas de API y los flujos de navegador seleccionados no demuestran por sí solos accesibilidad completa, comportamiento en todos los dispositivos, concurrencia de producción o ausencia de vulnerabilidades. La revisión visual del frontend y la revisión del PDF se registran separadamente. La ejecución de una futura CI debe enlazarse a un resultado real del repositorio.

Capítulo 9

Estructura, mantenimiento y refactorización

9.1. Situación inicial real

En la referencia inicial no había implementación ejecutable que refactorizar. Por ello no se fabrica un listado anterior, un defecto histórico, una versión V2 ni una prueba de regresión sobre código ausente. La separación de rutas y servicios durante esta construcción es una decisión del primer incremento.

9.2. Problema que se busca evitar

Si un controlador mezclara validación HTTP, reglas de acceso, transiciones y SQL, sería difícil probar una regla sin simular toda la solicitud y difícil cambiar el acceso a datos. La arquitectura reduce ese riesgo con módulos explícitos. Esta motivación no constituye evidencia de que ese controlador monolítico haya existido en el repositorio.

La conexión se recibe por inyección; los servicios contienen las reglas; las rutas traducen solicitudes y respuestas. No se introduce una jerarquía de clases por preferencia. La cohesión se evalúa por responsabilidades y consumidores, no por el número de archivos.

9.3. Protocolo para una refactorización futura

Una tarea de V2 deberá registrar:

1. Problema observado, archivo, función y referencia Git real.
2. Comportamiento que debe conservarse y pruebas ejecutadas antes del cambio.
3. Fragmento anterior obtenido de la referencia preservada, no reconstruido de memoria.
4. Cambio concreto, principio o patrón pertinente y razón para elegirlo.
5. Fragmento refactorizado y pruebas de regresión posteriores.
6. Resultado, limitaciones y revisión humana efectivamente registrada.

Si se añade una capacidad, como notificaciones por inscripción, se clasifica como ampliación funcional; no como refactorización pura. Los eventos de éxito deben ocurrir después del commit y solo se justifican si existe un consumidor real.

9.4. Strategy, Adapter y evolución

Strategy puede tener sentido si el cliente confirma diferentes algoritmos de evaluación o certificación. Adapter puede aislar una integración de proveedor externo. Ninguno se implementa solo para mostrarlo en el informe. La eventual migración PostgreSQL exige cambios comprobados y no se resuelve mediante una interfaz vacía.

La prioridad de mantenimiento es corregir defectos reales al encontrarlos. No se preserva un error deliberadamente para producir una historia académica de mejora.

Capítulo 10

Resultados y limitaciones

10.1. Artefactos preparados

La sesión construye el primer incremento local y el conjunto documental: diagnóstico, requisitos, backlog de 28 historias, criterios de aceptación, trazabilidad, proceso, planificación, decisiones, diagramas y fuentes del informe. El alcance funcional de implementación se concentra en autenticación, usuarios, roles y capacitaciones.

La evidencia de ejecución se lee en la tabla 8.2. Un archivo creado, una ruta escrita o un botón visible no prueban por sí solos que el flujo esté verificado. Los resultados finales deben indicar los comandos efectivamente completados y sus limitaciones.

10.2. Resultado esperado del flujo V1

El administrador de demostración puede trabajar con cuentas ficticias y capacitaciones almacenadas en SQLite a través de una API protegida. La interfaz presenta carga, confirmación y error según la respuesta real. El acceso y la edición dependen de los permisos actuales, no de un rol escrito manualmente en el navegador.

Se preservan las ramas de integrantes y los documentos previos. No se presenta la construcción local como una integración, publicación, revisión de compañeros o aprobación de Paz Sur. No se crean tags V1–V4 para simular una evolución que no ocurrió.

10.3. Limitaciones del producto

- No están implementados los flujos completos de participantes, asistencia, sesiones, resultados o historial.
- No se emiten certificados, ni se presupone que todas las capacitaciones los necesitan.
- Los catálogos de marcas, campañas, sedes y áreas son diseño candidato.
- Los reportes consolidados y las alertas no se presentan como disponibles por existir un permiso de catálogo.
- La matriz de roles, estados y reglas académicas necesita validación con el cliente.
- No se ha demostrado capacidad de producción, alta disponibilidad o cumplimiento normativo específico.

10.4. Limitaciones del entorno y del proceso

La falta de espacio en la unidad del perfil complicó la ejecución de herramientas. Se verificó disponibilidad de Node y SQLite. XeLaTeX se ejecutó realmente con instalación automática deshabilitada y falló por ausencia de `xelatex.fmt`; no pudo completar la generación del formato. Se preparó la alternativa pdfLaTeX con fuentes Latin Modern vectoriales y se corrigió la interacción entre Babel español y TikZ. El resultado de esa compilación se consigna en la tabla de evidencia, separado de la revisión visual del PDF.

Los Mermaid se revisaron como fuentes, pero su renderizado con un motor Mermaid no está verificado. El informe incluye esas fuentes y dos diagramas nativos TikZ; ambos tipos de artefacto deben distinguirse al presentar resultados.

Las ceremonias Scrum, las revisiones de compañeros y la integración en Git permanecen propuestas o pendientes hasta que existan registros. Las estimaciones de capacidad son propuestas y no reemplazan datos históricos de Jira.

10.5. Siguiente actividad recomendada

El equipo debe revisar el diff y ejecutar la demostración con datos ficticios, validar las reglas del primer incremento y registrar las observaciones. Después podrá seleccionar una tarea pequeña de participantes o instructores, verificar sus dependencias y completar sus criterios sin comprometer automáticamente el resto del producto.

Capítulo 11

Conclusiones

El desarrollo debe comenzar con un diagnóstico del árbol real. Los documentos previos aportaron un contrato útil, pero no demostraron la presencia de una aplicación en este clon. Preservar esa distinción permitió construir el primer incremento sin borrar el trabajo del equipo ni atribuirle evidencia inexistente.

La arquitectura de monolito modular separa interfaz, HTTP, reglas y persistencia con complejidad razonable para cuatro estudiantes. SQLite facilita la demostración local; PostgreSQL sigue siendo una decisión de evolución que requiere migración y pruebas reales.

El enfoque iterativo, incremental y adaptativo permite verificar un flujo pequeño antes de ampliar el alcance. Scrum, Kanban y prácticas de XP ofrecen una propuesta de trabajo; su aplicación efectiva requiere registros y responsabilidad humana.

La documentación de 28 historias no equivale a su implementación. La aceptación de cada historia, las verificaciones ejecutadas, la revisión de código y la integración determinan si un incremento cumple la Definition of Done.

El informe conserva decisiones, modelos, limitaciones y fragmentos importados del código. Debe actualizarse con cada módulo importante y con las pruebas realmente realizadas. La siguiente iteración debe partir del incremento revisado y de reglas comerciales confirmadas.

Bibliografía

[1] Agile Alliance. Extreme programming. https://agilealliance.org/glossary/xp/. Consultado el 12 de noviembre de 2026.

[2] Express.js. Moving to express 5. https://expressjs.com/en/guide/migrating-5/. Requisitos de entorno y migración; consultado el 12 de noviembre de 2026.

[3] Kanban Guides. The kanban guide, may 2025. https://kanbanguides.org/the-kanban-guide/2025.5/, 2025. Consultado el 12 de noviembre de 2026.

[4] Node.js. Crypto, node.js 24 api documentation. https://nodejs.org/docs/latest-v24.x/api/crypto.html. Consultado el 12 de noviembre de 2026.

[5] Node.js. Sqlite, node.js 24 api documentation. https://nodejs.org/download/release/v24.9.0/docs/api/sqlite.html. Referencia de DatabaseSync; la versión ejecutada en el entorno se registra por separado.

[6] OWASP Foundation. Cross-site request forgery prevention cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. Medidas de prevención; consultado el 12 de noviembre de 2026.

[7] Ken Schwaber and Jeff Sutherland. The scrum guide. https://scrumguides.org/scrum-guide.html, 2020. Consultado el 12 de noviembre de 2026.

[8] Vite. Getting started. https://vite.dev/guide/. Consultado el 12 de noviembre de 2026.

Apéndice A

Bitácora de desarrollo

Tabla A.1: Actividades documentadas en esta sesión

Fecha Lima	Módulo	Actividad, motivo y resultado
2026-11-12	Diagnóstico	Inspección del árbol y referencias Git. Base `cf6e720` con documentación y sin aplicación. Se preservan los antecedentes y se distingue otra carpeta descrita por los TXT.
2026-11-12	Arquitectura	Elaboración de `docs/ARQUITECTURA.md`, registro ADR y 11 fuentes Mermaid. Se diferencian nueve tablas V1 del modelo académico candidato.
2026-11-12	Planificación	Construcción de documentos de requisitos, aceptación, backlog, trazabilidad y proceso por el equipo de agentes. Sus estimaciones y responsabilidades propuestas no se atribuyen a aprobación humana.
2026-11-12	V1	Construcción asistida por IA de API, persistencia e interfaz de HU-01 a HU-04 y HU-16. Los archivos reales se detallan en el capítulo de implementación; las pruebas finales se registran en la tabla de evidencia.
2026-11-12	Entorno	Detección de falta de espacio en la unidad del perfil. El agente principal informó instalación de dependencias completada en el espacio disponible. Node 24.16.0 y SQLite 3.53.0 comprobados.
2026-11-12	Informe	Creación de `main.tex`, once capítulos, bibliografía y anexos. Escudo, facultad/escuela y nombres incompletos se dejan preparados para confirmación. No se inventa una imagen institucional.
2026-11-12	Calidad	Ejecución final de API, tipos, build y compilación LaTeX: ver tabla de evidencia, que debe actualizarse con resultados reales. Revisión humana e integración pendientes.

Fecha Lima	Módulo	Actividad, motivo y resultado
2026-11-12	Regresión final	`npm.cmd run check`: 28 pruebas aprobadas y build aprobado; Playwright: 4 pruebas Chrome aprobadas. Se corrigieron escalamiento directo e indirecto, migraciones incrementales, fecha Lima, límite de login y títulos móviles.
2026-11-12	Informe final	XeLaTeX no generó formato; pdfLaTeX con BibTeX compiló 50 páginas tras corregir Babel/TikZ, fuentes y portada. Capturas reales incluidas; revisión visual mediante páginas PNG y hojas de contacto. No se publicaron cambios.
2026-11-12	Alineación	Actualizados arquitectura, ADR y capítulos con permisos delegados, revocación, límite de login, migración idempotente y archivos de pruebas reales. Se diferencia consulta de instructores V1 de gestión futura.
2026-11-12	Compilación	XeLaTeX falló por formato ausente. En `main.tex` se añadió compatibilidad Babel–TikZ, anclas PDF no duplicadas y fuentes Latin Modern para el intento pdfLaTeX. El resultado final y la revisión visual se registran por separado.

A.1. Registro de futuras entradas

Cada entrada posterior debe indicar fecha, historia o tarea, archivos modificados, motivo, dificultad, comandos ejecutados, resultado, evidencia, revisión humana y pendiente. Cuando exista una refactorización, se conservarán referencias reales del antes y después y las pruebas correspondientes.

Este registro describe asistencia de IA. No asigna a Brenda, Adel, Franshesco o Daniela pruebas, escritura manual, ceremonias o revisiones que no hayan confirmado.

Apéndice B

Fuentes de los diagramas

Las fuentes distinguen V1 del diseño futuro. Los diagramas de inscripción y asistencia son modelos candidatos y no evidencia de endpoints ejecutados.

B.1. Arquitectura general

Fuente versionable: `docs/diagramas/arquitectura.mmd`.

```
flowchart TB
  %% V1 local; los nodos futuros son diseño candidato, no implementación.
  persona[Personal autorizado / trabajador]
  subgraph web[Presentación V1]
    ui[React + TypeScript]
    apiClient[Cliente HTTP /api]
    ui --> apiClient
  end
  subgraph backend[Aplicación Node 24 + Express 5]
    middleware[Sesión + CSRF + permisos + validación]
    routes[Rutas HTTP]
    users[Servicio de usuarios y roles]
    trainings[Servicio de capacitaciones]
    db[Conexión y transacciones]
    middleware --> routes
    routes --> users
    routes --> trainings
    users --> db
    trainings --> db
  end
  sqlite[(SQLite V1)]
  persona --> ui
  apiClient -->|JSON, cookie HttpOnly| middleware
  db -->|SQL parametrizado| sqlite
  futuro[Participantes, sesiones, asistencia,
  evaluaciones, historial y reportes: futuros]
  postgres[(PostgreSQL: decisión futura)]
  routes -. evolución candidata .-> futuro
  db -. migración pendiente .-> postgres
  classDef future fill:#fff6df,stroke:#a36d13,stroke-dasharray:5 5;
  class futuro,postgres future;
```

B.2. Casos de uso

Fuente versionable: `docs/diagramas/casos_uso.mmd`.

```
flowchart LR
  %% Aproximación de casos de uso con flowchart; V1 azul, candidatos amarillo, parciales
   violeta.
  admin[Administrador]
  coord[Coordinador]
  instructor[Instructor]
  participante[Participante]
  rrhh[RR. HH.]
  gerencia[Gerencia]
  subgraph sistema[Sistema de Gestión de Capacitaciones]
    login([HU-16 Iniciar sesión])
    users([HU-01 Registrar usuarios])
    roles([HU-02 Asignar roles y permisos])
    trainings([HU-03 / HU-04 Crear y modificar capacitación])
    assigned([HU-07 Consulta de asignadas: alcance parcial V1])
    people([HU-05 / HU-06 / HU-17 Instructor y participantes])
    teaching([HU-08 / HU-09 / HU-28
    Sesiones, asistencia y evaluación])
    learning([HU-10 / HU-11 / HU-12 / HU-18
    Inscripciones, resultados y certificados])
    history([HU-13 / HU-23 Historial y pendientes])
    reports([HU-14 / HU-15 / HU-24 / HU-25
    Reportes, estados y filtros])
    catalogs([HU-20 / HU-21 / HU-22
    Marcas, campañas, sedes y áreas])
    planning([HU-19 / HU-26 / HU-27
    Cupos, alertas y reprogramación])
  end
  admin --> login
  coord --> login
  instructor --> login
  participante --> login
  rrhh --> login
  gerencia --> login
  admin --> users
  admin --> roles
  admin --> trainings
  coord --> trainings
  instructor --> assigned
  coord -.-> people
  instructor -.-> teaching
  participante -.-> learning
  rrhh -.-> history
  gerencia -.-> reports
  coord -.-> catalogs
  coord -.-> planning
  classDef implemented fill:#e3f2ff,stroke:#17628d;
  classDef candidate fill:#fff6df,stroke:#a36d13,stroke-dasharray:5 5;
  classDef partial fill:#eee8ff,stroke:#71519e,stroke-dasharray:5 5;
  class login,users,roles,trainings implemented;
  class people,teaching,learning,history,reports,catalogs,planning candidate;
  class assigned partial;
```

B.3. Entidad–relación V1

Fuente versionable: `docs/diagramas/er_v1.mmd`.

```
erDiagram
  %% Modelo físico V1. Consultar 001_initial.sql para tipos/restricciones exactos.
  users {
    INTEGER id PK
    TEXT first_name
    TEXT last_name
    TEXT email UK "Único sin distinguir mayúsculas"
    TEXT document UK "Opcional"
    TEXT password_hash
    INTEGER active "Booleano 0/1"
    TEXT department
    TEXT specialty
    TEXT created_at
    TEXT updated_at
  }
  roles {
    INTEGER id PK
    TEXT code UK
    TEXT name
  }
  permissions {
    TEXT code PK
    TEXT name
  }
  user_roles {
    INTEGER user_id PK,FK
    INTEGER role_id PK,FK
  }
  role_permissions {
    INTEGER role_id PK,FK
    TEXT permission_code PK,FK
  }
  sessions_auth {
    TEXT token_hash PK
    INTEGER user_id FK
    TEXT csrf_token
    TEXT expires_at
  }
  trainings {
    INTEGER id PK
    TEXT code UK "Sin distinguir mayúsculas"
    TEXT title
    TEXT description
    TEXT start_date
    TEXT end_date
    TEXT start_time
    TEXT end_time
    TEXT modality
    TEXT location
    INTEGER capacity "Entre 1 y 10000"
    TEXT status
    INTEGER instructor_id FK "Opcional"
    INTEGER responsible_id FK
    TEXT created_at
    TEXT updated_at
  }
  audit_log {
    INTEGER id PK
    INTEGER user_id FK
    TEXT action
    TEXT entity
    INTEGER entity_id
    TEXT created_at
```

```
  }
  schema_migrations {
    TEXT version PK
    TEXT applied_at
  }
  users ||--o{ user_roles : recibe
  roles ||--o{ user_roles : asignado
  roles ||--o{ role_permissions : contiene
  permissions ||--o{ role_permissions : habilita
  users ||--o{ sessions_auth : autentica
  users o|--o{ trainings : instruye
  users ||--o{ trainings : responsable
  users o|--o{ audit_log : registra
```

B.4. Entidad–relación del producto candidato

Fuente versionable: `docs/diagramas/er_producto_candidato.mmd`.

```
erDiagram
  %% Diseño completo candidato: NO es el esquema migrado de V1.
  %% users, roles y trainings ya tienen representación V1; se omiten tablas RBAC auxiliares
   aquí.
  users {
    int id PK
    string email UK
    boolean active
  }
  roles {
    int id PK
    string code UK
  }
  workers {
    int id PK
    int user_id FK "Opcional y único"
    string employee_code UK
    string first_name
    string last_name
    int site_id FK
    int area_id FK
    boolean active
  }
  sites {
    int id PK
    string name UK
    string address
  }
  areas {
    int id PK
    string name UK
  }
  supplier_brands {
    int id PK
    string name UK
    boolean active
  }
  campaigns {
    int id PK
    string name
    date start_date
    date end_date
  }
```

```
trainings {
  int id PK
  string code UK
  string title
  date start_date
  date end_date
  int capacity
  string status
  int instructor_id FK "Usuario con rol instructor"
  int responsible_id FK
  int brand_id FK "Futuro opcional"
  int campaign_id FK "Futuro opcional"
  int site_id FK "Futuro opcional"
}
training_sessions {
  int id PK
  int training_id FK
  date date
  string title
}
enrollments {
  int id PK
  int training_id FK
  int participant_id FK "Usuario; inscripción sin cuenta pendiente"
  string status
  datetime created_at
}
attendance {
  int id PK
  int enrollment_id FK
  int session_id FK
  string status
  int recorded_by FK
}
evaluations {
  int id PK
  int enrollment_id FK,UK
  decimal grade
  string observation
  int recorded_by FK
}
certificates {
  int id PK
  int enrollment_id FK,UK
  string code UK
  string status
  datetime issued_at
  int issued_by FK
  string participant_name
  string training_title
}
notifications {
  int id PK
  int user_id FK
  int training_id FK
  string message
  datetime created_at
  datetime read_at "Opcional"
}
users }o--o{ roles : recibe
users o|--o| workers : acceso_opcional
sites ||--o{ workers : ubica
areas ||--o{ workers : agrupa
```

```
  users o|--o{ trainings : instructor
  users ||--o{ trainings : responsable
  supplier_brands o|--o{ trainings : relaciona
  campaigns o|--o{ trainings : contextualiza
  sites o|--o{ trainings : organiza
  trainings ||--o{ training_sessions : contiene
  trainings ||--o{ enrollments : recibe
  users ||--o{ enrollments : participante
  enrollments ||--o{ attendance : recibe
  training_sessions ||--o{ attendance : registra
  enrollments ||--o| evaluations : obtiene
  enrollments ||--o| certificates : puede_obtener
  users ||--o{ notifications : recibe
  trainings o|--o{ notifications : origina
```

B.5. Responsabilidades de módulos

Fuente versionable: `docs/diagramas/clases.mmd`.

```
classDiagram
  %% Vista conceptual de módulos y responsabilidades V1; la API usa funciones, no estas clases
    literales.
  class InterfazReact {
    mostrarFormulario()
    mostrarError()
    consultarAPI()
  }
  class RutasAuth {
    login()
    me()
    logout()
  }
  class RutasUsuarios {
    listar()
    crear()
    editar()
    asignarRoles()
  }
  class RutasCapacitaciones {
    listar()
    consultar()
    crear()
    editar()
  }
  class Seguridad {
    requireAuth()
    requirePermission()
    verificarCSRF()
    hashPassword()
  }
  class ServicioUsuarios {
    validarUsuario()
    conservarAdministradorActivo()
    guardarUsuario()
    guardarPermisos()
  }
  class ServicioCapacitaciones {
    validarPlanificacion()
    validarInstructor()
    comprobarTransicion()
    guardarCapacitacion()
```

```
  }
  class BaseDatos {
    createDatabase()
    transaction()
    aplicarMigraciones()
  }
  InterfazReact ..> RutasAuth : JSON
  InterfazReact ..> RutasUsuarios : JSON
  InterfazReact ..> RutasCapacitaciones : JSON
  RutasAuth ..> Seguridad
  RutasUsuarios ..> Seguridad
  RutasCapacitaciones ..> Seguridad
  RutasUsuarios --> ServicioUsuarios
  RutasCapacitaciones --> ServicioCapacitaciones
  ServicioUsuarios --> BaseDatos
  ServicioCapacitaciones --> BaseDatos
  Seguridad --> BaseDatos
```

B.6. Componentes

Fuente versionable: `docs/diagramas/componentes.mmd`.

```
flowchart TB
  subgraph navegador[Navegador]
    forms[Formularios React]
    session[Estado de sesión y CSRF]
    client[Cliente API]
    forms --> client
    session --> client
  end
  subgraph node[Proceso Node]
    app[Express app]
    auth[Router auth]
    users[Router usuarios / roles]
    trainings[Router capacitaciones]
    security[Seguridad y validación]
    userService[Servicio usuarios]
    trainingService[Servicio capacitaciones]
    db[Persistencia y migraciones]
    app --> auth
    app --> users
    app --> trainings
    auth --> security
    users --> security
    trainings --> security
    users --> userService
    trainings --> trainingService
    auth --> db
    security --> db
    userService --> db
    trainingService --> db
  end
  client -->|HTTP misma procedencia| app
  db --> archivo[(Archivo SQLite)]
  tests[Node Test Runner + Supertest]
  tests -->|createApp con base aislada| app
```

B.7. Registrar una capacitación

Fuente versionable: `docs/diagramas/secuencia_registrar_capacitacion.mmd`.

```
sequenceDiagram
  autonumber
  actor C as Coordinador o administrador
  participant UI as Formulario React
  participant API as Router capacitaciones
  participant S as Servicio capacitación
  participant DB as SQLite
  Note over UI,DB: Flujo V1; reglas de planificación provisionales documentadas
  C->>UI: Completa título, fechas, modalidad y cupo
  UI->>API: POST /api/trainings + cookie + X-CSRF-Token
  API->>API: Sesión, permiso trainings.manage y esquema
  alt Solicitud inválida o sin permiso
    API-->>UI: 401 / 403 / 422 con mensaje
  else Solicitud autorizada
    API->>S: Crear con actor autenticado
    S->>S: Comprobar fechas, código, instructor y estado inicial
    S->>DB: BEGIN IMMEDIATE
    S->>DB: INSERT trainings
    S->>DB: INSERT audit_log
    alt Restricción incumplida
      DB-->>S: Error de integridad
      S->>DB: ROLLBACK
      S-->>API: Conflicto o validación
      API-->>UI: 409 / 422
    else Guardado completo
      S->>DB: COMMIT
      S-->>API: Capacitación persistida
      API-->>UI: 201 con data
      UI-->>C: Confirmación y lista actualizada
    end
  end
```

B.8. Inscribir un participante: diseño futuro

Fuente versionable: `docs/diagramas/secuencia_inscribir_participante.mmd`.

```
sequenceDiagram
  autonumber
  actor C as Coordinador
  participant UI as Gestión de participantes
  participant API as API de inscripción
  participant S as Servicio inscripción
  participant DB as Base relacional
  Note over C,DB: DISEÑO CANDIDATO: HU-06 y HU-19 pendientes; no implementado en V1
  C->>UI: Selecciona capacitación y trabajador
  UI->>API: POST /api/trainings/:id/enrollments
  API->>API: Autenticar, CSRF, permiso y esquema
  API->>S: Inscribir participante
  S->>DB: Iniciar transacción con exclusión de escritura
  S->>DB: Consultar capacitación, persona, duplicado y cupo
  alt Estado cerrado, duplicado, inactivo o sin cupo
    S->>DB: ROLLBACK
    S-->>API: Error de regla de negocio
    API-->>UI: 404 / 409 / 422 según caso
  else Datos válidos
    S->>DB: INSERT inscripción + auditoría
    S->>DB: COMMIT
```

```
    S-->>API: Inscripción persistida
    API-->>UI: 201 con data
    UI-->>C: Lista y cupos actualizados
  end
  Note over S,DB: Un evento o aviso futuro solo se origina después del commit
```

B.9. Registrar asistencia: diseño futuro

Fuente versionable: `docs/diagramas/secuencia_registrar_asistencia.mmd`.

```
sequenceDiagram
  autonumber
  actor I as Instructor asignado
  participant UI as Control de asistencia
  participant API as API de sesiones
  participant S as Servicio asistencia
  participant DB as Base relacional
  Note over I,DB: DISEÑO CANDIDATO: HU-08 y HU-28 pendientes; no implementado en V1
  I->>UI: Selecciona sesión y marcas de asistencia
  UI->>API: PUT /api/sessions/:id/attendance
  API->>API: Verificar sesión, CSRF y teaching.manage
  API->>S: Registrar lote y actor
  S->>DB: Consultar sesión, capacitación e instructor
  S->>S: Verificar alcance, estado y pertenencia de inscripciones
  alt Alguna inscripción o marca no es válida
    S-->>API: Rechazar lote completo
    API-->>UI: 403 / 409 / 422
  else Lote válido
    S->>DB: BEGIN
    loop Cada inscripción del lote
      S->>DB: Insertar o actualizar marca única por sesión e inscripción
    end
    S->>DB: Registrar auditoría y COMMIT
    S-->>API: Marcas guardadas
    API-->>UI: 200 con data
    UI-->>I: Confirmación tras persistencia
  end
```

B.10. Estados

Fuente versionable: `docs/diagramas/estados.mmd`.

```
stateDiagram-v2
  %% Grafo V1 provisional; necesita validación del cliente.
  [*] --> planificada : creación
  [*] --> programada : creación
  planificada --> programada : programar
  planificada --> cancelada : cancelar
  programada --> planificada : volver a planificación
  programada --> en_curso : iniciar
  programada --> cancelada : cancelar
  en_curso --> finalizada : cerrar
  en_curso --> cancelada : cancelar
  finalizada --> [*]
  cancelada --> [*]
  note right of planificada
    Edición manteniendo estado permitida
    en estados abiertos
```

```
  end note
  note right of finalizada
    Finalizadas y canceladas inmutables.
    Reapertura pendiente de validación.
  end note
```

B.11. Roles y permisos

Fuente versionable: `docs/diagramas/roles_permisos.mmd`.

```
flowchart LR
  %% Matriz inicial del contrato. Permisos editables salvo administrador.
  %% Los permisos futuros no implican que existan sus módulos en V1.
  admin[Administrador] --> all[Todos los permisos]
  coord[Coordinador] --> tm[trainings.manage]
  coord --> tv[trainings.view_all]
  coord --> pm[people.manage]
  coord --> cm[certificates.manage: módulo futuro]
  instructor[Instructor] --> teach[teaching.manage: consulta de asignadas V1, gestión futura]
  participant[Participante] --> learn[learning.view: módulo futuro]
  hr[RR. HH.] --> history[history.view: módulo futuro]
  hr --> tv
  hr --> reports[reports.view: módulo futuro]
  management[Gerencia] --> reports
  management --> tv
  all --> um[users.manage]
  all --> rm[roles.manage]
  all --> tm
  all --> tv
  all --> pm
  all --> cm
  all --> teach
  all --> learn
  all --> history
  all --> reports
```